

LeetCode

Interview Preparation Guide

Java Solutions with Worked Examples

10
Easy

10
Medium

5
Hard

- ✓ Line-by-line code with syntax highlights
- ✓ Step-by-step example traces for every problem
- ✓ Time & Space complexity for each solution
- ✓ Concise approach summary before each solution
- ✓ Covers Easy → Medium → Hard progression

Table of Contents

■ Easy Problems

- E1. Two Sum — $O(n)$
- E2. Valid Parentheses — $O(n)$
- E3. Maximum Subarray (Kadane's) — $O(n)$
- E4. Climbing Stairs — $O(n)$
- E5. Best Time to Buy and Sell Stock — $O(n)$
- E6. Valid Palindrome — $O(n)$
- E7. Missing Number — $O(n)$
- E8. Single Number — $O(n)$
- E9. Contains Duplicate — $O(n)$
- E10. Move Zeroes — $O(n)$

■ Medium Problems

- M1. Longest Substring Without Repeating Chars — $O(n)$
- M2. Longest Palindromic Substring — $O(n^2)$
- M3. Add Two Numbers (Linked List) — $O(\max(n,m))$
- M4. 3Sum — $O(n^2)$
- M5. Container With Most Water — $O(n)$
- M6. Search in Rotated Sorted Array — $O(\log n)$
- M7. Generate Parentheses — $O(4^n / \sqrt{n})$
- M8. Combination Sum — $O(2^n)$ worst
- M9. Trapping Rain Water — $O(n)$
- M10. Group Anagrams — $O(n \cdot k \cdot \log k)$ — $k = \text{max word length}$

■ Hard Problems

- H1. Median of Two Sorted Arrays — $O(\log \min(m,n))$
- H2. Regular Expression Matching — $O(m \cdot n)$
- H3. Merge k Sorted Lists — $O(n \log k)$ — $n = \text{total nodes}$
- H4. First Missing Positive — $O(n)$
- H5. Substring with Concatenation of All Words — $O(n \cdot m)$ — $n = |s|, m = |\text{words}|$

Reading Guide

■ Approach	Core strategy summary
■ Example Trace	Step-by-step walkthrough
■ Time / ■ Space	Complexity analysis

■ EASY PROBLEMS

#E1 Two Sum

Given an array of integers and a target, return indices of the two numbers that add up to target. Each input has exactly one solution and you may not use the same element twice.

■ *HashMap lookup: store each value's index; for every element check if its complement (target – num) already exists in the map.*

■ Example Trace

```
Input : nums = [2, 7, 11, 15], target = 9
Step 1: i=0, num=2 → complement=7 → map empty → store {2:0}
Step 2: i=1, num=7 → complement=2 → found at 0 → return [0,1]
Output: [0, 1] ✓
```

Java Solution

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (map.containsKey(complement))
            return new int[]{ map.get(complement), i };
        map.put(nums[i], i);
    }
    throw new IllegalArgumentException("No solution");
}
```

■ Time: $O(n)$

■ Space: $O(n)$

#E2 Valid Parentheses

Given a string containing only '(', ')', '{', '}', '[', ']', determine if the string is valid. Every open bracket must be closed by the same type and in the correct order.

■ *Stack: push opening brackets; on a closing bracket pop and verify it matches. Valid iff stack is empty at end.*

■ Example Trace

```
Input : "{[()]}"
Push: { → stack=[{]
Push: [ → stack=[{, []
Push: ( → stack=[{, [, (]
Pop: ) matches ( → stack=[{, []
Pop: ] matches [ → stack=[{]
Pop: } matches { → stack=[] → return true ✓
```

Java Solution

```
public boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c=='(' || c=='{' || c=='[') { stack.push(c); }
        else {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if ((c==')' && top!='(') ||
                (c=='}' && top!='{') ||
                (c==']' && top!='[')) return false;
        }
    }
    return stack.isEmpty();
}
```

■ Time: **O(n)**

■ Space: **O(n)**

#E3 Maximum Subarray (Kadane's)

Find the contiguous subarray (containing at least one number) which has the largest sum and return its sum.

■ *Kadane's Algorithm: at each position decide whether to extend the current subarray or start fresh. Track global maximum throughout.*

■ Example Trace

```
Input : [-2, 1, -3, 4, -1, 2, 1, -5, 4]
i=1: maxEnd=max(1, -2+1)=1, maxSoFar=1
i=3: maxEnd=max(4, 1-3+4)=4, maxSoFar=4
i=5: maxEnd=max(2, 4-1+2)=5, maxSoFar=5
i=6: maxEnd=max(1, 5+1)=6, maxSoFar=6
Output: 6 (subarray [4,-1,2,1]) ✓
```

Java Solution

```
public int maxSubArray(int[] nums) {
    int maxSoFar = nums[0];
    int maxEndingHere = nums[0];
    for (int i = 1; i < nums.length; i++) {
        maxEndingHere = Math.max(nums[i], maxEndingHere + nums[i]);
        maxSoFar = Math.max(maxSoFar, maxEndingHere);
    }
    return maxSoFar;
}
```

■ Time: $O(n)$

■ Space: $O(1)$

#E4 Climbing Stairs

You can climb 1 or 2 steps at a time. In how many distinct ways can you reach the top of n stairs?

■ *Dynamic programming (Fibonacci pattern):* $ways(n) = ways(n-1) + ways(n-2)$. Use two variables instead of an array for $O(1)$ space.

■ Example Trace

```
n=5: ways[1]=1, ways[2]=2
i=3: third=1+2=3, first=2, second=3
i=4: third=2+3=5, first=3, second=5
i=5: third=3+5=8, first=5, second=8
Output: 8 distinct ways ✓
```

Java Solution

```
public int climbStairs(int n) {
    if (n == 1) return 1;
    int first = 1, second = 2;
    for (int i = 3; i <= n; i++) {
        int third = first + second;
        first = second;
        second = third;
    }
    return second;
}
```

■ Time: $O(n)$

■ Space: $O(1)$

#E5 Best Time to Buy and Sell Stock

Given prices[], find the maximum profit from one buy-sell transaction. If no profit is possible, return 0.

■ *Single pass: track the minimum price seen so far. At each day compute profit = price – minPrice and update the maximum profit.*

■ Example Trace

```
Input : [7, 1, 5, 3, 6, 4]
Day 1: price=7, min=7, profit=0
Day 2: price=1, min=1, profit=0
Day 3: price=5, min=1, profit=4
Day 5: price=6, min=1, profit=5 ← max
Output: 5 ✓
```

Java Solution

```
public int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE;
    int maxProfit = 0;
    for (int price : prices) {
        if (price < minPrice) minPrice = price;
        else if (price - minPrice > maxProfit)
            maxProfit = price - minPrice;
    }
    return maxProfit;
}
```

■ Time: **O(n)**

■ Space: **O(1)**

#E6 Valid Palindrome

Given string s, determine if it is a palindrome considering only alphanumeric characters and ignoring case.

■ *Two pointers from both ends, skip non-alphanumeric chars, compare lowercased characters until pointers cross.*

■ Example Trace

```
Input : "A man, a plan, a canal: Panama"
After filtering: "amanaplanacanalpanama"
l=0 'a' == r=19 'a' → move inward
l=1 'm' == r=18 'm' → move inward
...all characters match...
Output: true ✓
```

Java Solution

```
public boolean isPalindrome(String s) {
    int l = 0, r = s.length() - 1;
    while (l < r) {
        while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
        while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
        if (Character.toLowerCase(s.charAt(l)) !=
            Character.toLowerCase(s.charAt(r))) return false;
        l++; r--;
    }
    return true;
}
```

■ Time: $O(n)$

■ Space: $O(1)$

#E7 Missing Number

Given array containing n distinct numbers in [0,n], find the missing number.

■ *Gauss sum trick: expected = $n*(n+1)/2$. Subtract actual array sum to get the missing value. No extra space needed.*

■ Example Trace

```
Input : [3, 0, 1] n=3
expected = 3*4/2 = 6
actual = 3+0+1 = 4
missing = 6-4 = 2 ✓
```

Java Solution

```
public int missingNumber(int[] nums) {
    int n = nums.length;
    int expected = n * (n + 1) / 2;
    int actual = 0;
    for (int num : nums) actual += num;
    return expected - actual;
}
```

■ Time: $O(n)$

■ Space: $O(1)$

#E8 Single Number

Every element appears twice except for one. Find the single element in $O(n)$ time and $O(1)$ space.

■ *XOR property: $a \oplus a = 0$ and $a \oplus 0 = a$. XOR-ing all numbers cancels pairs, leaving only the single element.*

■ Example Trace

```
Input : [4, 1, 2, 1, 2]
result = 0^4 = 4
result = 4^1 = 5
result = 5^2 = 7
result = 7^1 = 6
result = 6^2 = 4 ← the single number ✓
```

Java Solution

```
public int singleNumber(int[] nums) {
    int result = 0;
    for (int num : nums) result ^= num;
    return result;
}
```

■ Time: $O(n)$

■ Space: $O(1)$

#E9 Contains Duplicate

Return true if any value appears at least twice in the array.

■ *HashSet: attempt to add each element. If add() returns false, the element was already present — duplicate found.*

■ Example Trace

```
Input : [1, 2, 3, 1]
add(1) → true, set={1}
add(2) → true, set={1,2}
add(3) → true, set={1,2,3}
add(1) → false → duplicate! → return true ✓
```

Java Solution

```
public boolean containsDuplicate(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int num : nums)
        if (!set.add(num)) return true;
    return false;
}
```

■ Time: $O(n)$

■ Space: $O(n)$

#E10 Move Zeroes

Move all 0's to end while maintaining relative order of non-zero elements. Do it in-place.

■ *Two-pointer write: iterate once placing non-zeros at 'writePos'; then fill remaining indices with zeros.*

■ Example Trace

```
Input : [0, 1, 0, 3, 12]
Pass 1 (non-zeros): pos=0→1, 1→3, 3→12 → [1,3,12,3,12]
Pass 2 (fill zeros): [1,3,12,0,0]
Output: [1, 3, 12, 0, 0] ✓
```

Java Solution

```
public void moveZeroes(int[] nums) {
    int pos = 0;
    for (int num : nums)
        if (num != 0) nums[pos++] = num;
    while (pos < nums.length) nums[pos++] = 0;
}
```

■ Time: $O(n)$

■ Space: $O(1)$

■ MEDIUM PROBLEMS

#M1 Longest Substring Without Repeating Chars

Find the length of the longest substring without repeating characters.

■ *Sliding window with HashSet: expand right pointer; when duplicate found shrink left until window is valid again.*

■ Example Trace

```
Input : "abcabcbb"
r=0: add a → window="a", len=1
r=1: add b → window="ab", len=2
r=2: add c → window="abc", len=3
r=3: 'a' exists → remove a, l=1 → add a → window="bca", len=3
Output: 3 ("abc" or "bca") ✓
```

Java Solution

```
public int lengthOfLongestSubstring(String s) {
    Set<Character> set = new HashSet<>();
    int left = 0, maxlen = 0;
    for (int r = 0; r < s.length(); r++) {
        while (set.contains(s.charAt(r))) {
            set.remove(s.charAt(left++));
        }
        set.add(s.charAt(r));
        maxlen = Math.max(maxlen, r - left + 1);
    }
    return maxlen;
}
```

■ Time: $O(n)$

■ Space: $O(\min(n,m))$ — m =charset size

#M2 Longest Palindromic Substring

Given string s, return the longest palindromic substring.

■ *Expand-around-center: for each index try odd-length (i,i) and even-length (i,i+1) centers. Track the widest expansion seen.*

■ Example Trace

```
Input : "babad"
i=1 ('a'): expand odd → 'bab' len=3, start=0, end=2
i=2 ('b'): expand odd → 'aba' len=3, same length, no update
Output: "bab" ✓ (or "aba")
```

Java Solution

```
public String longestPalindrome(String s) {
    int start = 0, end = 0;
    for (int i = 0; i < s.length(); i++) {
        int len = Math.max(expand(s,i,i), expand(s,i,i+1));
        if (len > end - start) {
            start = i - (len-1)/2;
            end = i + len/2;
        }
    }
    return s.substring(start, end+1);
}

private int expand(String s, int l, int r) {
    while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r))
        { l--; r++; }
    return r - l - 1;
}
```

■ Time: $O(n^2)$

■ Space: $O(1)$

#M3 Add Two Numbers (Linked List)

Two non-empty linked lists represent non-negative integers stored in reverse order. Add them and return the sum as a linked list.

■ *Simulate addition digit-by-digit with a carry variable. Use a dummy head node to simplify edge cases.*

■ Example Trace

```
l1=[2→4→3] (342) l2=[5→6→4] (465)
Step 1: 2+5+0=7, carry=0 → node 7
Step 2: 4+6+0=10, carry=1 → node 0
Step 3: 3+4+1=8, carry=0 → node 8
Output: 7→0→8 (807 = 342+465) ✓
```

Java Solution

```
public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0), cur = dummy;
    int carry = 0;
    while (l1!=null || l2!=null || carry!=0) {
        int sum = carry;
        if (l1!=null) { sum+=l1.val; l1=l1.next; }
        if (l2!=null) { sum+=l2.val; l2=l2.next; }
        carry = sum/10;
        cur.next = new ListNode(sum%10);
        cur = cur.next;
    }
    return dummy.next;
}
```

■ Time: $O(\max(n,m))$

■ Space: $O(\max(n,m))$

#M4 3Sum

Return all unique triplets [a,b,c] from the array such that $a+b+c=0$.

■ Sort array; fix first element i ; apply two-pointer on the rest. Skip duplicates at all three levels to avoid repeat triplets.

■ Example Trace

```
Input : [-1, 0, 1, 2, -1, -4] → sorted: [-4, -1, -1, 0, 1, 2]
i=0 (-4): l=1, r=5 → -4-1+2=-3<0 → l++
i=1 (-1): l=2, r=5 → -1-1+2=0 → triplet [-1, -1, 2] ✓
i=1 (-1): l=3, r=4 → -1+0+1=0 → triplet [-1, 0, 1] ✓
Output: [[-1, -1, 2], [-1, 0, 1]] ✓
```

Java Solution

```
public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();
    for (int i=0; i<nums.length-2; i++) {
        if (i>0 && nums[i]==nums[i-1]) continue;
        int l=i+1, r=nums.length-1;
        while (l < r) {
            int sum = nums[i]+nums[l]+nums[r];
            if (sum==0) {
                res.add(Arrays.asList(nums[i], nums[l], nums[r]));
                while (l<r && nums[l]==nums[l+1]) l++;
                while (l<r && nums[r]==nums[r-1]) r--;
                l++; r--;
            } else if (sum<0) l++;
            else r--;
        }
    }
    return res;
}
```

■ Time: $O(n^2)$

■ Space: $O(1)$ excluding output

#M5 Container With Most Water

Given heights $h[]$, find two lines that together with x-axis form a container holding the most water.

■ *Two pointers from ends. Area = width $\times$ $\min(h[l], h[r])$. Always move the pointer with the smaller height inward to potentially improve.*

■ Example Trace

```
Input : [1,8,6,2,5,4,8,3,7]
l=0(h=1), r=8(h=7): area=1×1=1, move l (smaller)
l=1(h=8), r=8(h=7): area=7×7=49, move r
l=1(h=8), r=6(h=8): area=5×8=40
Best area found: 49 ✓
```

Java Solution

```
public int maxArea(int[] height) {
    int l=0, r=height.length-1, max=0;
    while (l < r) {
        max = Math.max(max, (r-l)*Math.min(height[l],height[r]));
        if (height[l] < height[r]) l++;
        else r--;
    }
    return max;
}
```

■ Time: $O(n)$

■ Space: $O(1)$

#M6 Search in Rotated Sorted Array

Search for target in a rotated sorted array of distinct integers. Must run in $O(\log n)$.

■ *Modified binary search: one half is always sorted. Check which half is sorted, then determine whether target falls in it.*

■ Example Trace

```
Input : [4,5,6,7,0,1,2], target=0
mid=3 (7): left[4..7] sorted, 0 not in [4,7] → l=4
mid=5 (1): right[1..2] sorted, 0 not in (1,2] → r=4
mid=4 (0): nums[4]==0 → return 4 ✓
```

Java Solution

```
public int search(int[] nums, int target) {
    int l=0, r=nums.length-1;
    while (l <= r) {
        int mid = l + (r-l)/2;
        if (nums[mid]==target) return mid;
        if (nums[l] <= nums[mid]) { // left half sorted
            if (target>=nums[l] && target<nums[mid]) r=mid-1;
            else l=mid+1;
        } else { // right half sorted
            if (target>nums[mid] && target<=nums[r]) l=mid+1;
            else r=mid-1;
        }
    }
    return -1;
}
```

■ Time: $O(\log n)$

■ Space: $O(1)$

#M7 Generate Parentheses

Given n pairs of parentheses, generate all combinations of well-formed parentheses.

■ **Backtracking:** add '(' while $open < n$; add ')' while $close < open$. Base case: $string\ length == 2n$.

■ Example Trace

```
n=2: Decision tree
 "(" → "(" → "(" → "(" → "()" ✓
 "(" → "(" → "()" → "()" ✓
 "(" → "()" → "()" → "()" ✓
Output: ["()", "()()"] ✓
```

Java Solution

```
public List<String> generateParenthesis(int n) {
    List<String> res = new ArrayList<>();
    backtrack(res, new StringBuilder(), 0, 0, n);
    return res;
}

private void backtrack(List<String> res, StringBuilder sb,
    int open, int close, int max) {
    if (sb.length() == max*2) { res.add(sb.toString()); return; }
    if (open < max) { sb.append('('); backtrack(res, sb, open+1, close, max); sb.deleteCharAt(sb.length()-1); }
    if (close < open){ sb.append(')'); backtrack(res, sb, open, close+1, max); sb.deleteCharAt(sb.length()-1); }
}
```

■ Time: $O(4^n / \sqrt{n})$

■ Space: $O(4^n / \sqrt{n})$

#M8 Combination Sum

Given distinct integers candidates and target, find all unique combinations that sum to target. The same number may be used unlimited times.

■ *Backtracking: for each candidate recursively reduce remain; pass same index i to allow reuse. Prune when remain < 0.*

■ Example Trace

```
candidates=[2,3,6,7], target=7
Path 2→2→2→(2>1 prune)
Path 2→2→3=7 ✓ [2,2,3]
Path 2→3→(2<2 prune)...etc
Path 7=7 ✓ [7]
Output: [[2,2,3],[7]] ✓
```

Java Solution

```
public List<List<Integer>> combinationSum(int[] cand, int target) {
    List<List<Integer>> res = new ArrayList<>();
    Arrays.sort(cand);
    bt(res, new ArrayList<>(), cand, target, 0);
    return res;
}
private void bt(List<List<Integer>> res, List<Integer> cur,
    int[] cand, int remain, int start) {
    if (remain == 0) { res.add(new ArrayList<>(cur)); return; }
    for (int i=start; i<cand.length; i++) {
        if (cand[i] > remain) break;
        cur.add(cand[i]);
        bt(res, cur, cand, remain-cand[i], i);
        cur.remove(cur.size()-1);
    }
}
```

■ Time: $O(2^{\text{min_candidate}})$ worst

■ Space: $O(\text{target}/\text{min_candidate})$ stack depth

#M9 Trapping Rain Water

Given elevation map heights[], compute total water trapped after raining.

■ *Two pointers with running max: process from the side with smaller height; trapped water at that bar = sideMax – height[i].*

■ Example Trace

```
Input : [0,1,0,2,1,0,1,3,2,1,2,1]
l=2(h=0): lMax=1, water+=1-0=1
l=4(h=1): lMax=2, water+=2-1=1
l=5(h=0): lMax=2, water+=2-0=2
...total water = 6 ✓
```

Java Solution

```
public int trap(int[] h) {
    int l=0, r=h.length-1, lMax=0, rMax=0, water=0;
    while (l < r) {
        if (h[l] < h[r]) {
            if (h[l] >= lMax) lMax=h[l];
            else water += lMax - h[l];
            l++;
        } else {
            if (h[r] >= rMax) rMax=h[r];
            else water += rMax - h[r];
            r--;
        }
    }
    return water;
}
```

■ Time: **O(n)**

■ Space: **O(1)**

#M10 Group Anagrams

Group strings that are anagrams of each other.

■ *Sorted-string as key: sort each word's characters to produce a canonical key; group original words under the same key in a HashMap.*

■ Example Trace

```
Input : ["eat","tea","tan","ate","nat","bat"]
"eat" → sorted "aet" → key "aet"
"tea" → sorted "aet" → key "aet" (same group)
"ate" → sorted "aet" → key "aet" (same group)
"tan" → sorted "ant", "nat" → sorted "ant" (same group)
Output: [["eat","tea","ate"],["tan","nat"],["bat"]] ✓
```

Java Solution

```
public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        char[] ch = s.toCharArray();
        Arrays.sort(ch);
        String key = new String(ch);
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}
```

■ Time: $O(n \cdot k \cdot \log k)$ — $k = \text{max word length}$

■ Space: $O(n \cdot k)$

■ HARD PROBLEMS

#H1 Median of Two Sorted Arrays

Find the median of two sorted arrays nums1 and nums2 in $O(\log(\min(m,n)))$ time.

■ *Binary search on the smaller array to find the correct partition where every element on the left side $\leq$ every element on the right side of the combined array.*

■ Example Trace

```
A=[1,3] B=[2,4] combined=[1,2,3,4] median=2.5
pA=1, pB=1: maxLA=1,minRA=3, maxLB=2,minRB=4
1<=4 and 2<=3 → correct partition
even length → (max(1,2)+min(3,4))/2 = (2+3)/2 = 2.5 ✓
```

Java Solution

```
public double findMedianSortedArrays(int[] A, int[] B) {
    if (A.length > B.length) return findMedianSortedArrays(B,A);
    int m=A.length, n=B.length, l=0, r=m;
    while (l <= r) {
        int pA=(l+r)/2, pB=(m+n+1)/2-pA;
        int maxLA = pA==0 ? Integer.MIN_VALUE : A[pA-1];
        int minRA = pA==m ? Integer.MAX_VALUE : A[pA];
        int maxLB = pB==0 ? Integer.MIN_VALUE : B[pB-1];
        int minRB = pB==n ? Integer.MAX_VALUE : B[pB];
        if (maxLA<=minRB && maxLB<=minRA) {
            if ((m+n)%2==0)
                return (Math.max(maxLA,maxLB)+Math.min(minRA,minRB))/2.0;
            return Math.max(maxLA,maxLB);
        } else if (maxLA > minRB) r=pA-1;
        else l=pA+1;
    }
    throw new RuntimeException();
}
```

■ Time: $O(\log \min(m,n))$

■ Space: $O(1)$

#H2 Regular Expression Matching

Implement regex matching supporting '.' (any single char) and '*' (zero or more of preceding element).

■ 2D DP: $dp[i][j] = \text{true}$ if $s[0..i-1]$ matches $p[0..j-1]$. Handle '*' by either using zero occurrences ($dp[i][j-2]$) or one-more ($dp[i-1][j]$).

■ Example Trace

```
s="aab" p="c*a*b"
dp[0][2]: p[1]='*' → dp[0][0]=true ('c*' matches empty)
dp[1][3]: p[2]='a', s[0]='a' → match → dp[0][2]=true
dp[2][3]: p[2]='a', s[1]='a' → match → dp[1][2]=true
dp[3][5]: p[4]='b', s[2]='b' → match → dp[2][4]=true
Output: true ✓
```

Java Solution

```
public boolean isMatch(String s, String p) {
    int m=s.length(), n=p.length();
    boolean[][] dp = new boolean[m+1][n+1];
    dp[0][0] = true;
    for (int j=1; j<=n; j++)
        if (p.charAt(j-1)=='*') dp[0][j]=dp[0][j-2];
    for (int i=1; i<=m; i++) for (int j=1; j<=n; j++) {
        char pc=p.charAt(j-1);
        if (pc=='.' || pc==s.charAt(i-1))
            dp[i][j]=dp[i-1][j-1];
        else if (pc=='*') {
            dp[i][j]=dp[i][j-2]; // zero of preceding
            char prev=p.charAt(j-2);
            if (prev=='.' || prev==s.charAt(i-1))
                dp[i][j]=dp[i-1][j]; // one or more
        }
    }
    return dp[m][n];
}
```

■ Time: $O(m \cdot n)$

■ Space: $O(m \cdot n)$

#H3 Merge k Sorted Lists

Merge k sorted linked lists into one sorted linked list.

■ *Divide and conquer: pair-wise merge reduces k lists to 1 in $O(\log k)$ rounds. Each round is $O(n) \rightarrow$ total $O(n \log k)$.*

■ Example Trace

```
lists = [1→4→5, 1→3→4, 2→6]
Round 1 (pairs): merge(L0,L1)→1→1→3→4→4→5, L2 unchanged
Round 2 (final): merge(above, L2)→1→1→2→3→4→4→5→6
Output: 1→1→2→3→4→4→5→6 ✓
```

Java Solution

```
public ListNode mergeKLists(ListNode[] lists) {
    if (lists==null||lists.length==0) return null;
    return mergeRange(lists, 0, lists.length-1);
}
private ListNode mergeRange(ListNode[] lists, int s, int e) {
    if (s==e) return lists[s];
    int mid = s+(e-s)/2;
    ListNode l = mergeRange(lists,s,mid);
    ListNode r = mergeRange(lists,mid+1,e);
    return merge2(l,r);
}
private ListNode merge2(ListNode a, ListNode b) {
    if (a==null) return b;
    if (b==null) return a;
    if (a.val<=b.val) { a.next=merge2(a.next,b); return a; }
    b.next=merge2(a,b.next); return b;
}
```

■ Time: $O(n \log k)$ — n =total nodes

■ Space: $O(\log k)$ recursion stack

#H4 First Missing Positive

Given unsorted array, find the smallest missing positive integer in $O(n)$ time and $O(1)$ space.

■ *Index as hash: place each positive number $x \in [1, n]$ at index $x-1$ via swapping. Then scan for first index where $\text{nums}[i] \neq i+1$.*

■ Example Trace

```
Input : [3, 4, -1, 1] n=4
Place phase: 3→idx2, 4→idx3, -1(skip), 1→idx0
Array after: [1, -1, 3, 4]
Scan: idx0→1 ✓, idx1→-1≠2 → return 2 ✓
```

Java Solution

```
public int firstMissingPositive(int[] nums) {
    int n = nums.length;
    for (int i=0; i<n; i++)
        while (nums[i]>0 && nums[i]<=n && nums[nums[i]-1]!=nums[i])
            swap(nums, i, nums[i]-1);
    for (int i=0; i<n; i++)
        if (nums[i]!=i+1) return i+1;
    return n+1;
}
private void swap(int[] a, int i, int j)
{ int t=a[i]; a[i]=a[j]; a[j]=t; }
```

■ Time: $O(n)$

■ Space: $O(1)$

#H5 Substring with Concatenation of All Words

Find all starting indices of substrings in *s* that are concatenations of all words (each used exactly once).

■ *Sliding window per offset: for each of wordLen offsets use two-pointer/hashmap approach to check each word-aligned window in $O(n)$ time.*

■ Example Trace

```
s="barfoothefoobarman", words=["foo","bar"]
wordLen=3, total=6
i=0: "bar"+"foo" → both in words, counts match → index 0 ✓
i=9: "foo"+"bar" → both in words, counts match → index 9 ✓
Output: [0, 9] ✓
```

Java Solution

```
public List<Integer> findSubstring(String s, String[] words) {
    List<Integer> res = new ArrayList<>();
    if (s==null||words==null||words.length==0) return res;
    int wl=words[0].length(), wc=words.length, total=wl*wc;
    Map<String,Integer> need = new HashMap<>();
    for (String w:words) need.merge(w,1,Integer::sum);
    for (int i=0; i<=s.length()-total; i++) {
        Map<String,Integer> seen = new HashMap<>();
        int j=0;
        for (; j<wc; j++) {
            String w=s.substring(i+j*wl, i+j*wl+wl);
            if (!need.containsKey(w)) break;
            seen.merge(w,1,Integer::sum);
            if (seen.get(w)>need.get(w)) break;
        }
        if (j==wc) res.add(i);
    }
    return res;
}
```

■ Time: $O(n \cdot m)$ — $n=|s|$, $m=|words|$

■ Space: $O(m)$

Quick Reference Cheat Sheet

Pattern	When to Use	Key Data Structures	Typical Complexity
Two Pointers	Sorted arrays, palindromes, pair sums	Array indices	$O(n)$
Sliding Window	Contiguous subarray/substring	HashSet / HashMap	$O(n)$
HashMap / HashSet	Lookup, duplicates, anagrams	HashMap, HashSet	$O(n)$
Binary Search	Sorted data, $O(\log n)$ required	Array + lo/hi pointers	$O(\log n)$
Stack	Matching brackets, monotone stack	Stack / Deque	$O(n)$
Backtracking	Permutations, combinations, subsets	Recursion + list	$O(n!)$ or 2^n
Dynamic Programming	Overlapping subproblems, optimal substructure	1D / 2D array	$O(n^2)$ typical
Divide & Conquer	Merge sort, binary search trees	Recursion	$O(n \log n)$
Greedy	Locally optimal $\rightarrow$ globally optimal	Sorted array	$O(n \log n)$
XOR Tricks	Find single number, missing number	Integer variable	$O(n)$
Recursion / DFS	Trees, graphs, nested structures	Call stack	$O(n)$ or $O(h)$